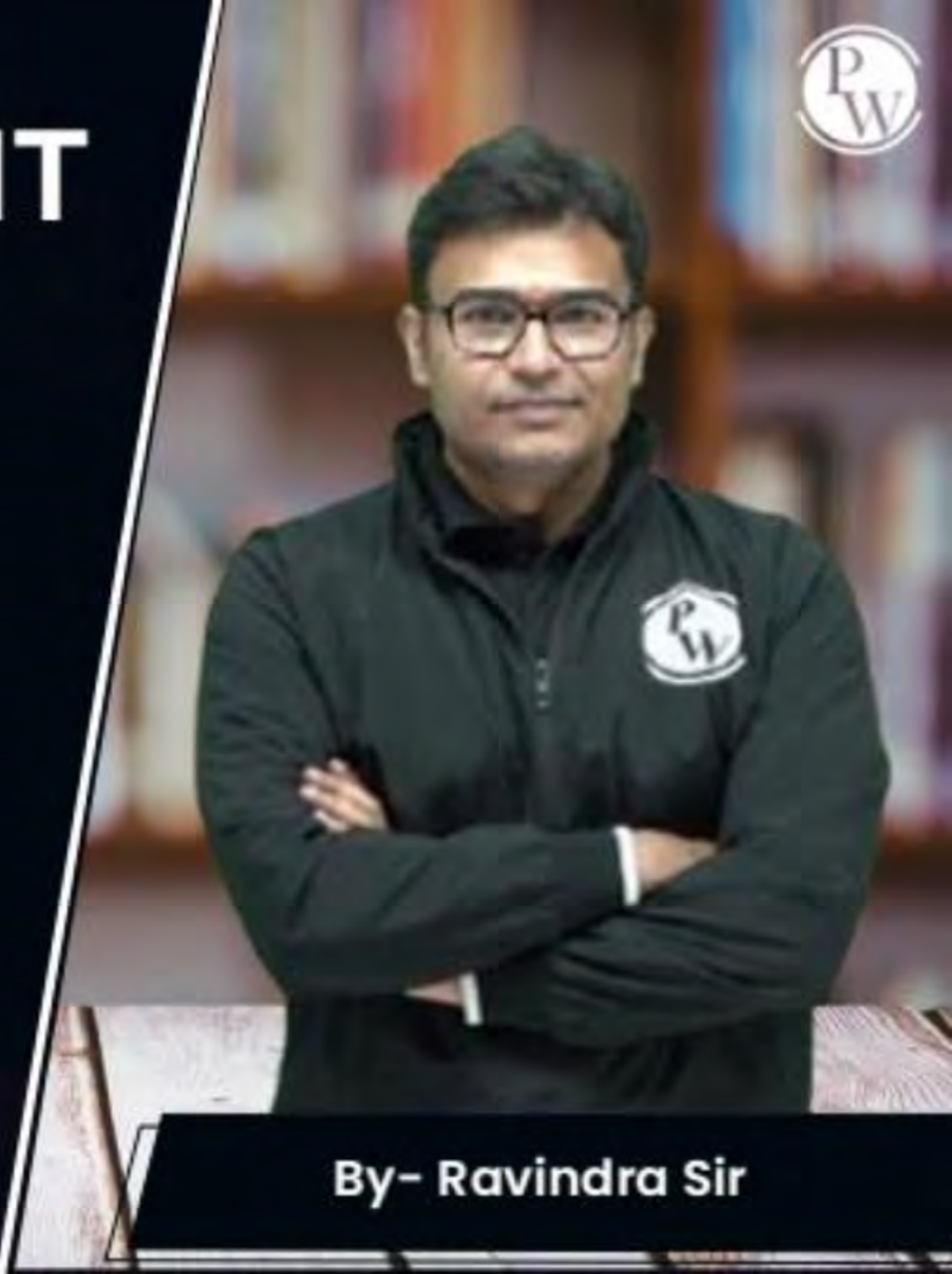
Computer Science & IT

COMPILER DESIGN
(CD)

Parsers

Lecture No. 9



By- Ravindra Sir

Recap of Previous Lecture



- Topic — Parsers
- Topic
- Topic

Topics to be Co

Topic

Topic

T

Parsel (OPP)



Join Telegram

@RBRSIRPW





Extra Ordinary Individuals: Stories to Ignite Student Motivation

Wilma Rudolph – The Girl Who Couldn't Walk, Then Outran the World

1. Background: Born in Tennessee, weak and underweight. Contracted polio as a child.

2. Struggles: Wore leg braces for years. Told she would never walk again — especially not run.

3. Achievements: Became the first American woman to win three Olympic golds in a single Games (1960).

4. Impact: She turned pain into propulsion. If she could outrun fate, you can outrun failure — with hustle, not hope.



Extra Ordinary Individuals: Stories to Ignite Student Motivation

Sunil Wadhwani – Technology Tycoon from IIT Madras

1. Background: Born in Delhi to a middle-class family; came without industry pedigree or investor backing.

2. Education: BTech from IIT Madras, followed by MS at Carnegie Mellon University (USA) Wikipedia.

3. Career Achievements: Co-founded IGATE, later Mastech Digital; scaled it into a global IT services giant serving Fortune 500 clients.

4. Impact: Helped build a world-class Indian IT services firm from modest beginnings. His leadership bridged the global service economy and Indian tech talent



Extra Ordinary Individuals: Stories to Ignite Student Motivation

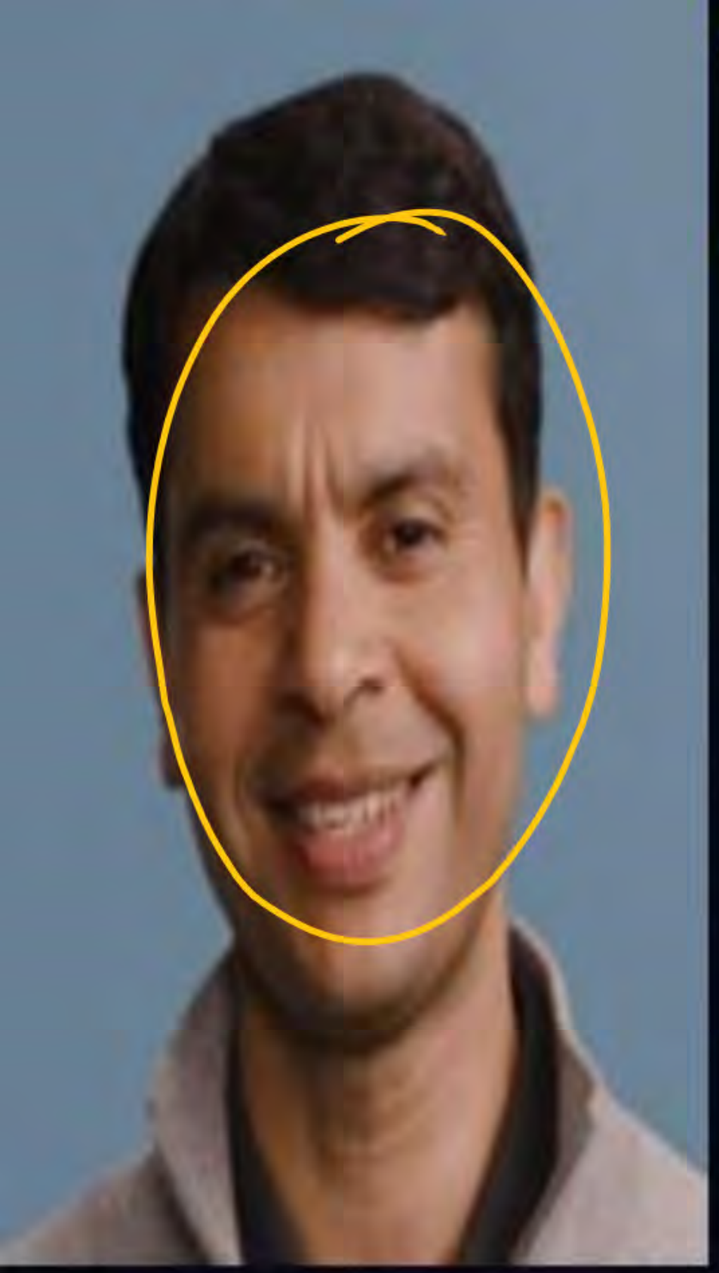
Giles Corey – The Man Who Died Saying "More Weight"

1. Background: 81-year-old farmer accused of witchcraft in 1692 Salem.

2. Struggles: Refused to plead guilty or innocent, knowing both would cost him truth and legacy. Crushed with stones over 3 days.

3. Achievements: Never uttered a false word. Protected his family's land through silence.

4. Impact: His quiet defiance outlived the mob. If he could hold his ground under stone, you can hold yours under stress.



Extra Ordinary Individuals: Stories to Ignite Student Motivation

Mohit Aron – Building Cutting-Edge Data Infrastructure

1. Background: Grew up in Chandigarh; pursued computer science amidst limited local tech opportunities.

2. Education: BTech in Computer Science IIT Delhi (1995), followed by MS and PhD in Computer Science from Rice University, USA (1998, 2001).

3. Career Achievements: Co-founder & CTO of Nutanix, pioneer of hyper-converged infrastructure; later founded Cohesity, a leading data management unicorn.

4. Impact: Revolutionized enterprise storage architecture and data resilience; from IIT Delhi to global tech leadership in cloud storage innovation

Operator precedence parser :-

To Construct operator precedence parser, the grammar should be operator grammar

Operator grammar :- The grammar doesn't contain null productions and two adjacent non terminals on the RHS of any productions is called operator grammar

$E \rightarrow EBE / id$

$B \rightarrow + / - / =$

$\Downarrow$ operator grammar

$E \rightarrow E+E / E-E / E=E / id$

The resulting grammar is ambiguous grammar but still we can build OPP. OPP doesn't bother whether a grammar is ambiguous or not. But the grammar must be operator grammar



OPP is the only parser that can accept some ambiguous grammars

All other parsers (LL, LR) are constructed only with unambiguous grammars.

→ $S \to SAS/a$

$A \to bSb/b$

⇓ operator

$S \to SbSbS/SbS/a$

↓ operator grammar

→ OPP mainly works by considering precedence among the operators.
That is why it is called operator precedence parser

→ Though the given grammar is ambiguous, the parser presents the the actual precedence among the operators in a table called precedence relation table

→ In this tables the actual precedence among two operators are presented as precedence relations $<, >, \doteq$

Precedence relations;

$a \gtrdot b$ ('a' takes precedence over 'b') So 'a' is reduced before 'b'

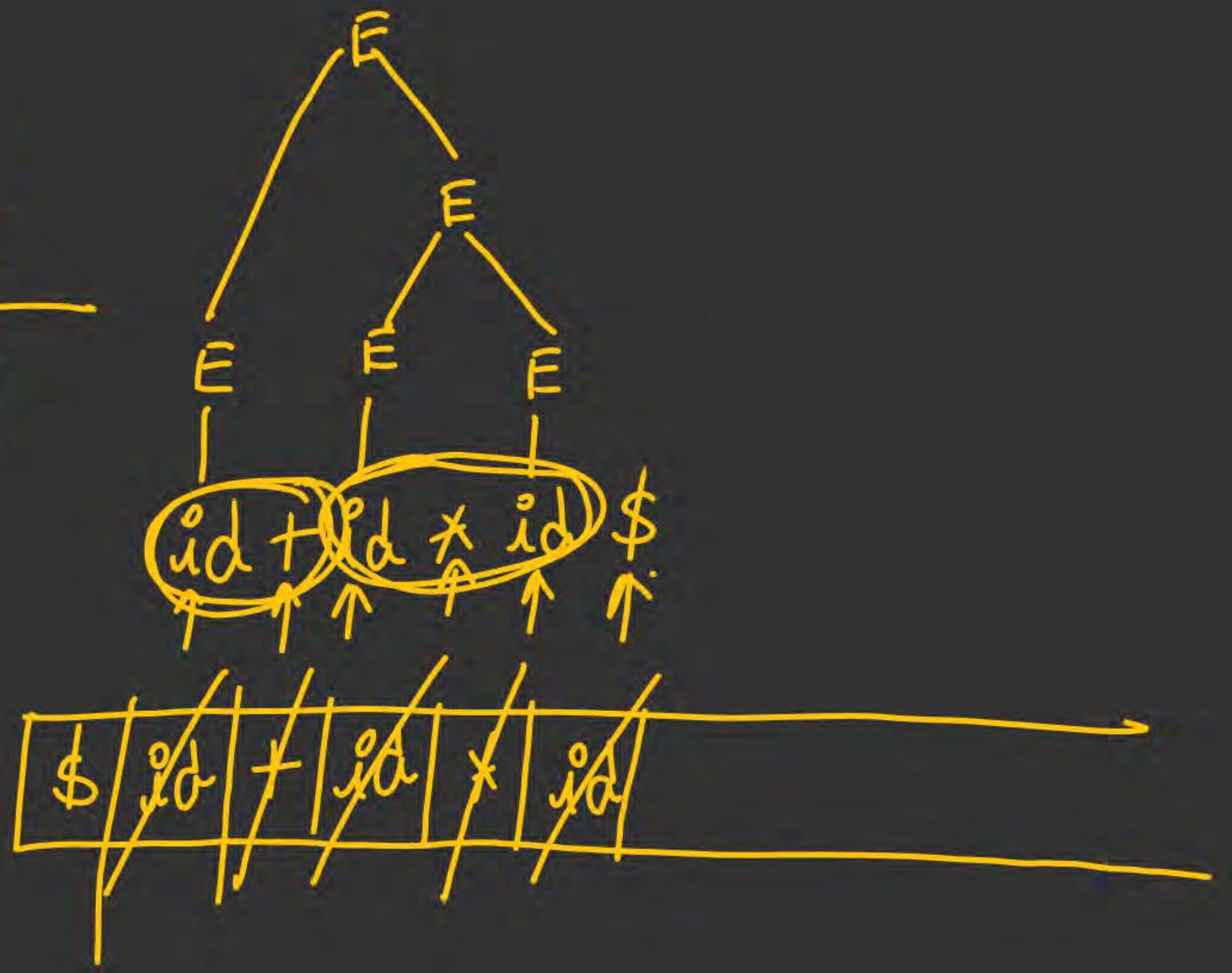
$a \lessdot b$ (b takes precedence over a) So 'b' is reduced before 'a'

$a \doteq b$ (a has same precedence as b) So both are reduced at the same time

E → E+E / E*E / id → ambiguous → operator grammar

Precedence relation table

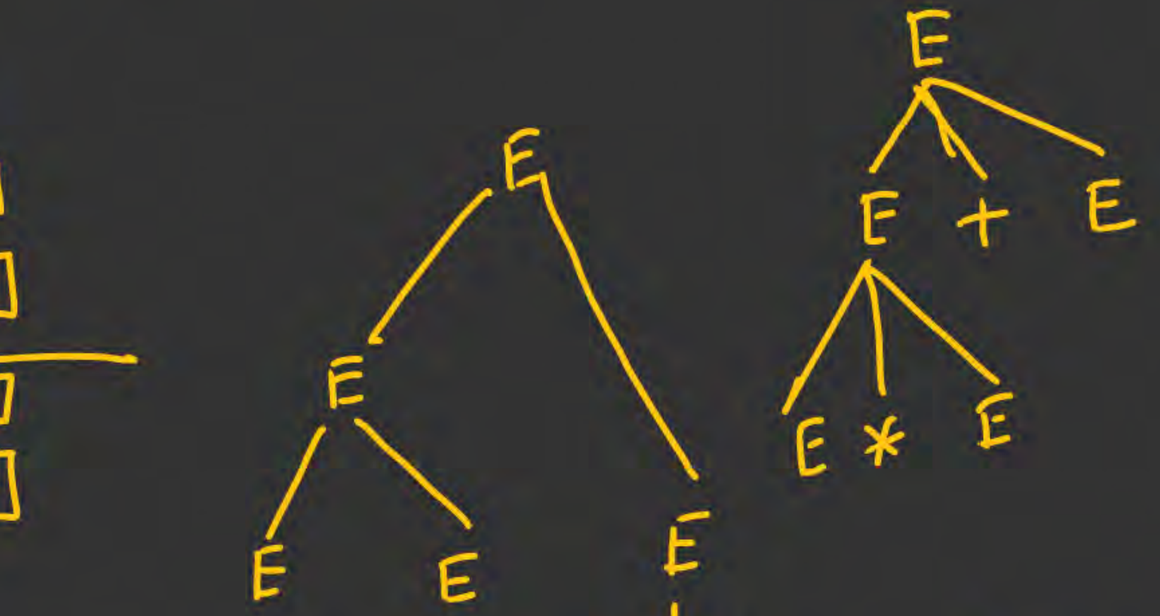
	id	+	*	$
id	—	⋗	⋗	⋗
+	⋖	⋗	⋖	⋗
*	⋖	⋗	⋗	⋗
$	⋖	⋖	⋖	—



E → E+E / E*E / id → ambiguous → operator grammar

Precedence relation table

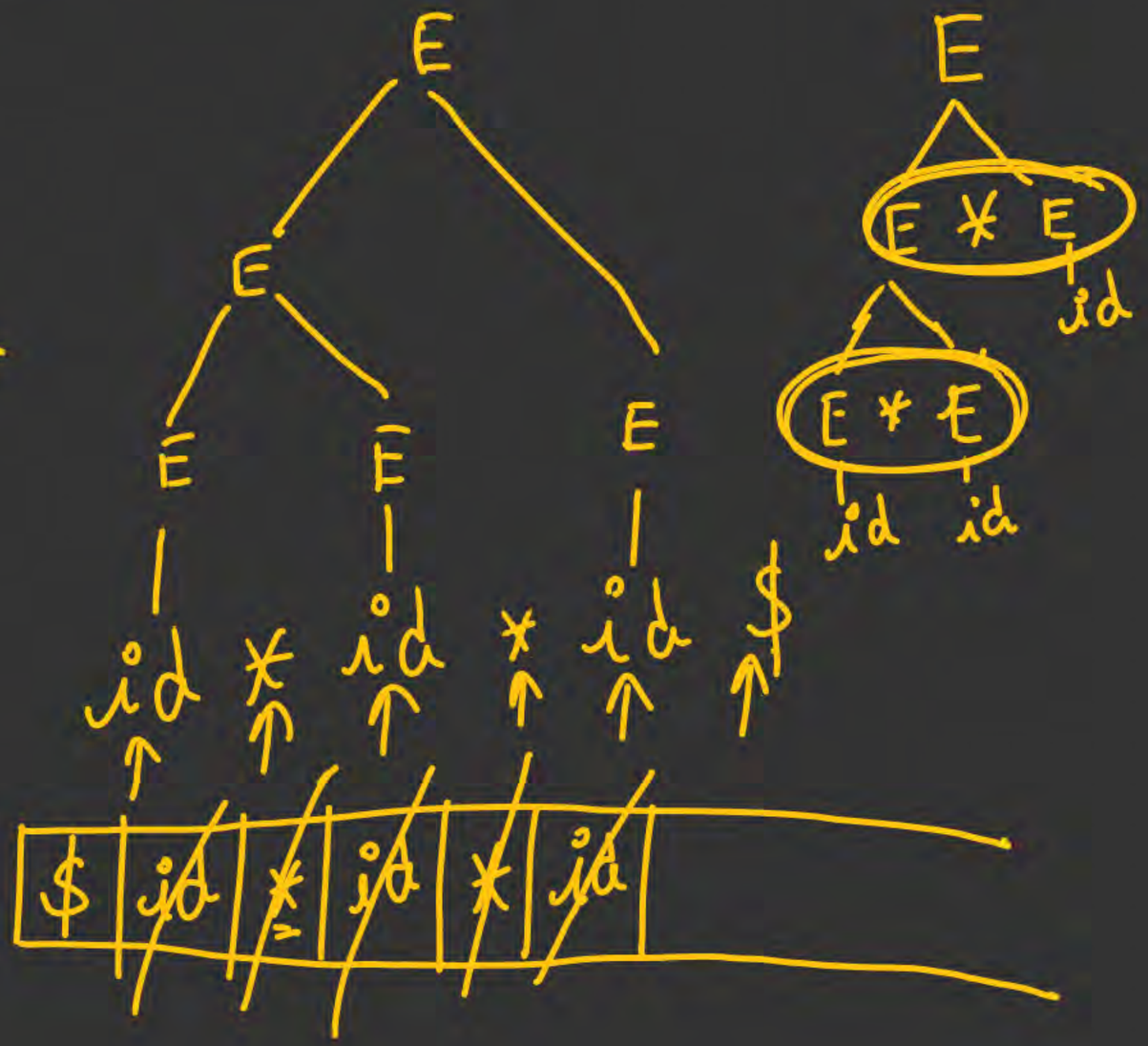
	id	+	*	$
id	—	>	>	>
+	<	>	<	>
*	<	>	>	>
$	<	<	<	—


E
E
E

E → E+E / E*E / id → ambiguous → operator grammar

Precedence relation table

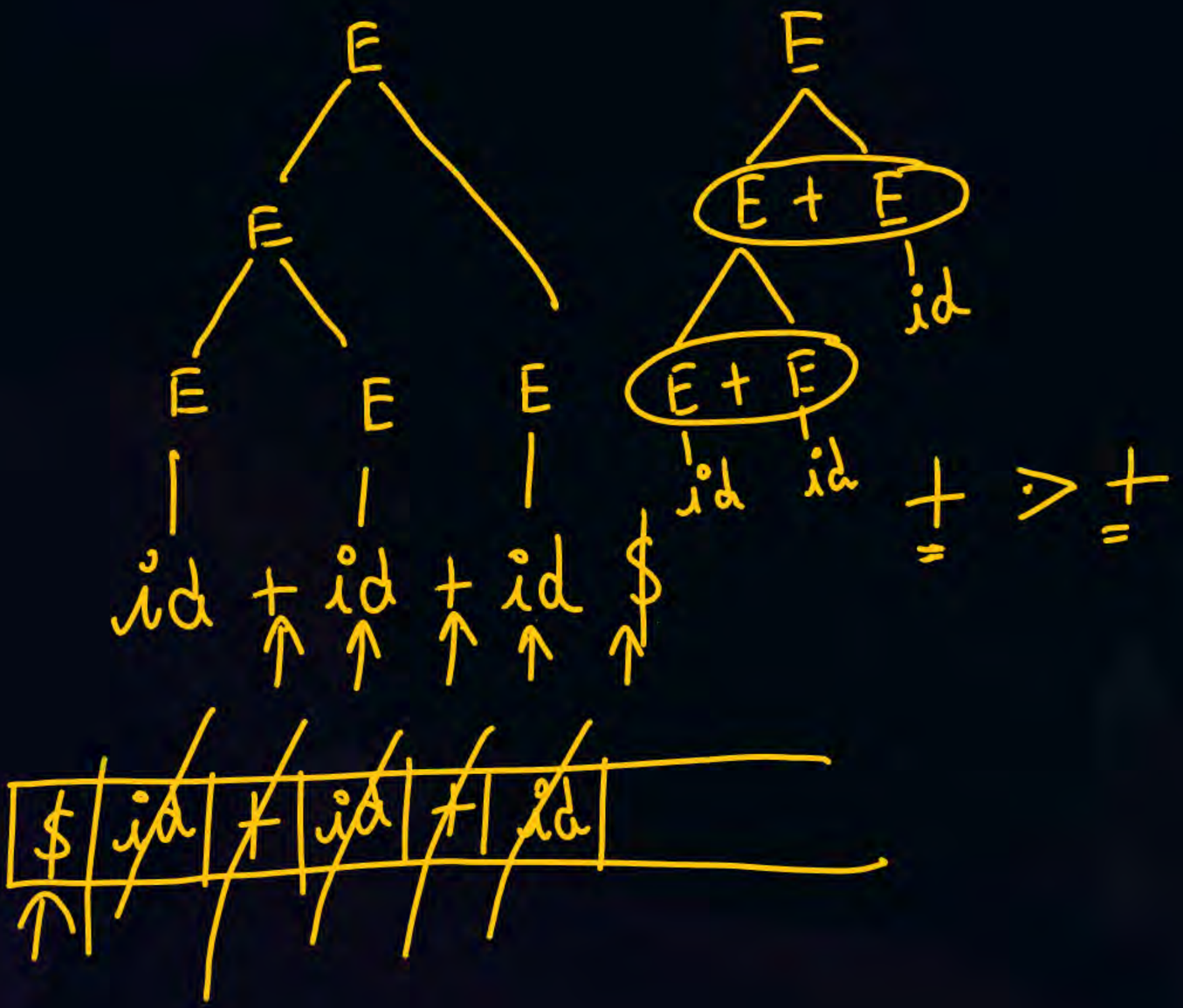
	id	+	*	$
id	—	⋗	⋗	⋗
+	⋖	⋗	⋖	⋗
*	⋖	⋗	⋗	⋗
$	⋖	⋖	⋖	—



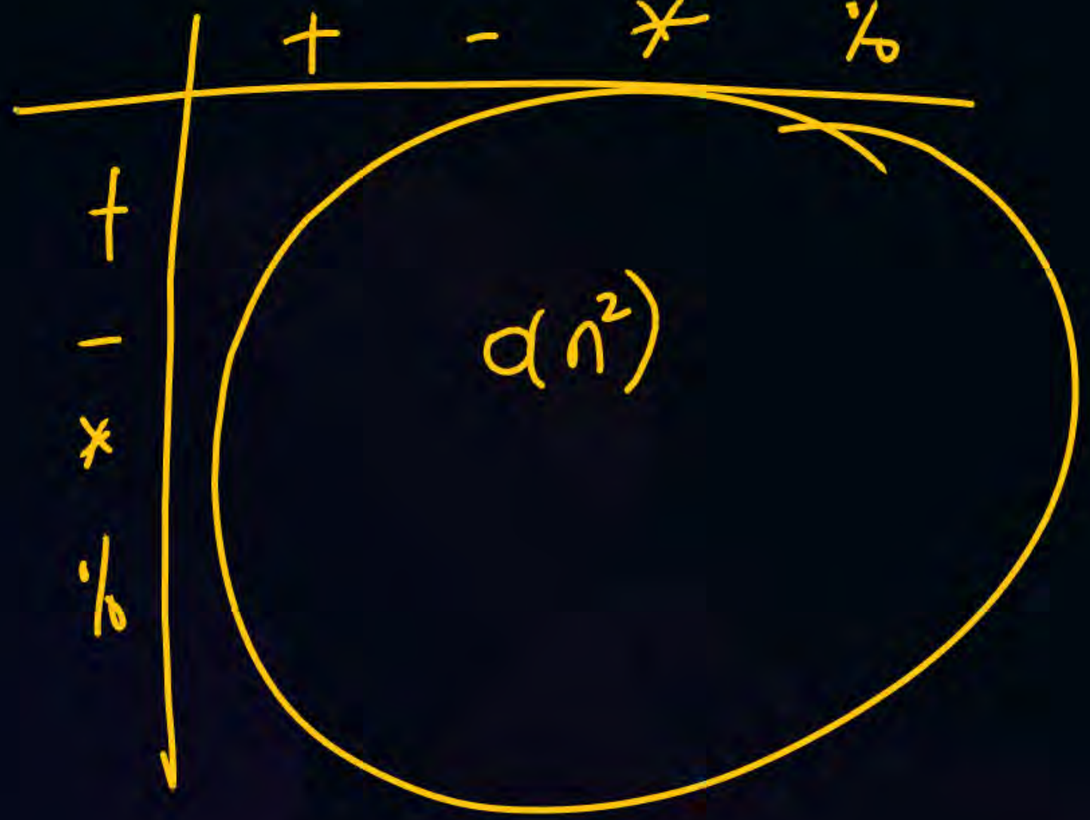
Parsing algorithm for OPP:-

1) Initially, Stack contains only $ and i/p buffer contain w$ where w is i/p string
2) Repeat forever
   a) let 'a' be the top element on the stack and 'b' is the current element pointed by the i/p pointer, that is look ahead symbol
   b) if $a \lessdot b$ or $a \doteq b$, push 'b' onto the stack and increment i/p pointer
   c) if $a \gtrdot b$, then (Reduce action) repeat pop the stack until top of the stack is $\lessdot$ b

d) if $a = b = \$$ announce Successful completion


E
E
E
E
E
E
E + E
E + E
id
id
id
id + id + id $
$ id + id + id

The disadvantage of operator relation table is that the size of table is $O(n^2)$, n is number of terms



$P \rightarrow SR/S \rightarrow ①$

$R \rightarrow bSR/bS \rightarrow ②$

$S \rightarrow wbS/w$

$w \rightarrow L*w/L$

$L \rightarrow id$

OG ⟹

Substitute ② in ①

$P \rightarrow Sb\underbrace{SR}_{P}/SbS/S$

$P \rightarrow SbP/SbS/S$

$S \rightarrow wbS/w$

$w \rightarrow L*w/L$

$L \rightarrow id$

a) Repeat

b) handsome

$P \to SbR/S \to (1)$

$XR \to bSR/bS \to (2)$

$S \to wbS/w$

$w \to L*w/L$

$L \to id$

Substitute (2) in (1)

$P \to S\,bSR/SbS/S$

$P \to SbP/SbS/S$

$S \to wbS/w$

$w \to L*w/L$

$L \to id$

✓

H/W

Precedence relation table

	id	*	b	$
id	—			
*				
b				
$				—

THANK - YOU